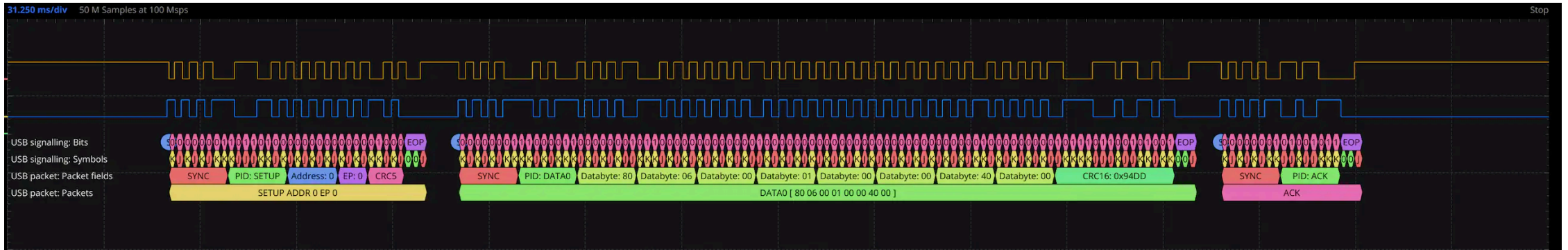


USB Part 2 (Internals)

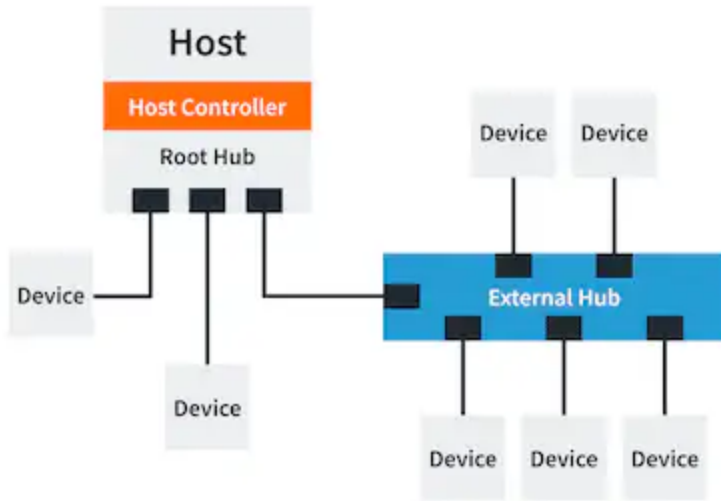


USB is more than just a cable — it's a complex communication protocol with multiple layers and concepts that work together to make our devices talk to each other seamlessly.

- USB Communication Model
- VID/PID/Class/Endpoints
- Addressing and Enumeration
- Data Transfer Types

USB Communication Model

USB communication follows a **host-to-device model**.



- The host (e.g., computer with Type A or C) initiates all communication and controls the data flow.
- Devices (e.g., mouse, keyboard, IoT board with Type B or C) respond to the host's requests but cannot initiate communication on their own.

Example: Your laptop (host) tells your mouse (device) "send me your position every 8ms"

- Mouse can't talk to printer directly
- Mouse can't initiate communication
- Mouse only responds when asked

VID/PID/Class: Identifying USB Devices

- USB devices are identified by:
 - Vendor ID (VID): Who made it?
 - Product ID (PID): What is it?
 - Device Class: What type of device is it?

VID/PID: Vendor ID and Product ID

Each USB device has a unique **vendor ID (VID)** and **product ID (PID)** assigned by the USB-IF organization.

- **Vendor ID (VID):** 16-bit number identifying the manufacturer (e.g., Logitech = 0x046D)
- **Product ID (PID):** 16-bit number identifying the specific product (e.g., Logitech mouse = 0xC52B)

USB Class

USB devices are grouped into **classes** so the OS knows which driver to load automatically.

Class	Code	Example Devices
HID	0x03	Keyboard, Mouse, Gamepad
Mass Storage	0x08	USB Flash Drive, External HDD
CDC	0x02	Serial Port, Arduino (via USB-Serial)
Audio	0x01	USB Headset, Microphone
Video	0x0E	Webcam
Hub	0x09	USB Hub

USB Endpoints

An **endpoint** is a communication channel on a USB device.

- Think of it as a **mailbox** on the device
- Each endpoint has a direction: **IN** (device → host) or **OUT** (host → device)
- **Endpoint 0** is always the **Control endpoint** (used for enumeration)
- A device can have up to **16 IN + 16 OUT** endpoints

A USB device has endpoint 0 for control, endpoint 1 for receiving commands, and endpoint 2 for sending sensor data:

Host	Device
----[OUT ep 1]---->	(receive commands)
<---[IN ep 2]-----	(send sensor data)
<-->[Control ep 0]	(setup / enumeration)

Real-World Example: Arduino Serial.print("Hi")

Serial.print("Hi") ← Application (you write this)



PIPE EP2 IN (Bulk Stream Pipe: Arduino → Host)



TRANSFER 2 bytes to send → fits in one transaction



TRANSACTION:

TOKEN → Host: "Device 5, EP2: give me data"

DATA0 ← Arduino: [0x48, 0x69] ('H', 'i')

ACK → Host: "Got it!"



PACKETS SYNC+TOKEN, SYNC+DATA0, SYNC+ACK



FRAME All 3 packets fit inside the current 1ms frame

Addressing and Enumeration

When you plug in a USB device, the host assigns it a unique **address** (0–127) through a process called **enumeration**.

Scenario: You plug in a USB flash drive

Time	Event
-----	-----
0ms	Device connects physically
1ms	Host detects voltage change on D+ or D- line
10ms	Host resets the port (100ms reset signal)
110ms	Host: "Hello, what are you?" (GET_DESCRIPTOR request)
111ms	Device: "I'm address 0, here's my info..."
112ms	Host reads: VID=0x0951, PID=0x1666 (Kingston DataTraveler)
113ms	Host: "Okay, from now on you're address 3"
114ms	Device: "Got it, I'm device #3 now"
115ms	Host loads driver: "mass storage class device"
200ms	Windows shows: "USB Drive (E:) is ready"

Data Transfer Types

Type	Guaranteed?	Speed	Use Case
Control	Yes	Low	Setup, enumeration
Bulk	No	High	File transfers
Interrupt	Yes (timing)	Medium	Keyboard, mouse
Isochronous	No (errors)	High	Audio, video ouse
Isochronous	No (errors)	High	Audio, video

Transfer Type 1: Control

Used for: Device setup and enumeration

- Always on Endpoint 0
- Small amounts of data
- Host asks, device responds
- Guaranteed delivery

Example: Host reads device descriptor during plug-in

```
Host    → "GET_DESCRIPTOR" request
Device  → sends 18-byte descriptor with VID/PID
Host    → "ACK"
```

Transfer Type 2: Bulk

Used for: Large data transfers where speed matters, errors not tolerated

- USB flash drives, printers
- No timing guarantee, but **error checking** with retry
- Fills leftover bandwidth

Transfer Type 3: Interrupt

Used for: Small, time-sensitive data (not a hardware interrupt!)

- Host **polls** the device at fixed intervals (e.g., every 8ms)
- Guaranteed maximum latency
- Perfect for keyboards and mice

Example: Mouse reports position every 8ms

Every 8ms:

```
Host   → "Any data, mouse?"      (poll)
Mouse  → "Moved: dx=+5, dy=-2"    (or "nothing")
```

Transfer Type 4: Isochronous

Used for: Real-time streaming where timing > accuracy

- Audio and video streams
- Fixed bandwidth reserved every frame (every 1ms)
- **No error correction** — a dropped audio sample is better than delay
- If data is lost, it's lost (no retry)

Analogy: A live phone call — you'd rather hear a small glitch than have the conversation freeze!

Every 1ms (USB frame):

Mic → sends audio chunk to host

[No ACK, no retry — just keep streaming]

Choosing the right transfer type is a key design decision in USB device firmware!

Arduino Nano Example

Arduino Nano uses a **USB-to-Serial bridge chip**:

- **CH340** (most clones, our boards)
- **FT232RL** (older / official)
- **CP2102** (some boards)

We need to install device drivers for these chips to communicate with the Arduino over USB.

Layer	Lives Where
USB Driver	PC / OS
USB Firmware	Bridge chip

So when Nano connects:

1. Bridge chip (Firmware) enumerates as CDC
2. OS loads CDC driver
3. Serial port appears

USB Device Class

When connected to a PC:

USB Class → CDC-ACM (Virtual COM Port)

Meaning:

- Appears as a Serial Port
- Used by Serial Monitor
- Used for sketch upload

Endpoint Structure (CDC)

Four endpoints (EP0 - 3):

Typical endpoint configuration:

Endpoint	Direction	Type	Function	Meaning
EP0	Control	Control	Enumeration / control	Device identification, line coding
EP1 OUT	Host → Device	Bulk	Serial RX	PC sends serial data
EP2 IN	Device → Host	Bulk	Serial TX	Arduino sends serial data
EP3 IN	Device → Host	Interrupt	Serial state notification	Device reports line status

DTR and EPO

DTR (Data Terminal Ready) signal is used to reset the Arduino during sketch upload:

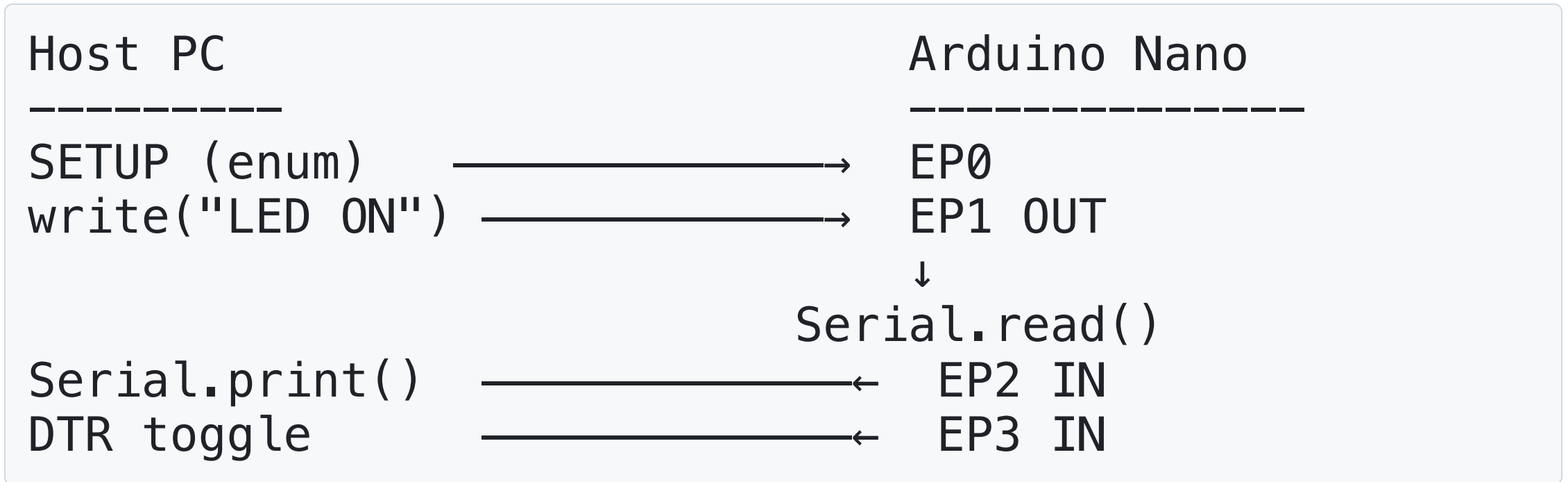
- DTR is originally from RS-232 serial communication.
- It is not TX/RX data, but a control signal that the host toggles to indicate "I'm ready to communicate".
- **DTR asserted (ON)** → "Terminal is ready"
- **DTR deasserted (OFF)** → "Terminal is not ready"

Computer (DTE)	<-->	Modem (DCE)
Computer (DTE)	<-->	Modem (DCE)

How Arduino Accesses USB Endpoints without knowing it?

1. ATmega328P has UART
2. It knows nothing about USB
3. UART <-> Bridge <-> EP1/EP2 carry data
4. EP0 handles enumeration + DTR control
5. EP3 is optional status notification — not required for reset (EP3 is for CDC class, but DTR is handled via EP0 control requests)

Communication Flow



Notice IN/OUT is from the perspective of the host, not the device (Arduino).

What Happens in Your Sketch

Code:

```
Serial.begin(9600);  
Serial.print("Hello");
```

Actual USB path:

```
ATmega328P UART  
  ↓  
USB-Serial Bridge (CH340 / FT232)  
  ↓  
Endpoint 2 IN (Bulk)  
  ↓  
Host PC Serial Driver
```